Python project MUST

1 Introduction

This document contains the description of the project for the course for the academic year 2025-2026.

The project consists of assignments located in the world of passwords: validation, determining password strength, and hashing.

2 Basic Information

A password is a secret sequence of characters (letters, numbers, symbols) used to prove that someone is who they claim to be. It is a security key that provides access to systems, files, and accounts.

Passwords are used for **authentication**, **access control**, and **data protection** to log in to a computer/laptop or smartphone, access an email account or social media (Facebook, Instagram, ...), etc.

2.1 Structure/Composition of a password

A (strong) password must meet several conditions:

- **Length**: at least 8 characters, preferably more.
- **Complexity**: a combination of uppercase letters, lowercase letters, numbers, and special characters.
  - at least 1 **digit** (`012...9`)
  - at least 1 **uppercase letter** (`AB...Z`)
  - at least 1 **special character**

  Commonly used special characters are:

$$!@\#\$\%\wedge\,\&*()-_=+[]\{\};:,.?/$$

- **Unpredictability**: no obvious words like `password`, your name, date of birth, or `123456`.
- **Unique**: use a different password for every account.
- **Difficult to guess, but memorable**: a sentence or abbreviation can be useful, e.g. `ILoveProgrammingInPython3.12!`

In principle, a password may contain any character that your keyboard (or even Unicode) can display. However, in practice, there are limitations depending on the system or application. In what follows, we limit ourselves to the 24 special characters listed above.

Note: a password can be a valid combination of uppercase, lowercase, numbers, and special characters and still be a **weak password**. Examples are variants of `password`, `p@ssword`, etc. The file `Weakpasswords.txt` contains a list of such variants.

3 Safe (strong) and unsafe (weak) passwords

Passwords can be assessed in terms of security. Typical criteria are the **strength score** S and the **entropy** H.

3.1 The strength score

A strength score S is a *heuristic* way to assess passwords - often used by websites or applications. The score S is the result of a points system where points are given for certain properties:

Property	**Points**
$8 \leq$ Length ≤ 12	+1
Length > 12	+2
Contains lowercase letters	+1
Contains uppercase letters	+1
Contains digits	+1
Contains special characters	+1
No common words	+2
No character repetition	+1 or -1

A list of common words can be found in the file `words.txt`.

To determine the score for the last property, we calculate the number of unique characters in proportion to the total length. If this ratio is less than **0.7**, a point is subtracted.

The interpretation of S is as follows:

- $S \leq 3$: weak
- $4 \leq S \leq 6$: fair
- $S \geq 7$: strong

Example

For the password `ILoveProgrammingInPython3.12!`, for example, we find that the length is greater than 12 (+2), it contains characters from every character set (+4), but it contains common words and the number of unique characters in proportion to the total length is less than 0.7 (-1). This results in a score of $S = 4 + 2 - 1 = 5$, and the password is thus labeled as `"fair"`.

3.2 Entropy

Entropy is a measure of the amount of *uncertainty* or *randomness* in a password — the higher the entropy, the harder it is to guess or crack.

The classic formula for entropy H is:

$$H = L \times \log_2(N)\,,$$

where L is the length of the password and N is the number of possible characters (lowercase, uppercase, ...) that can be used per position.

Character set	**Possible symbols**
Lowercase letters (a-z)	26
Uppercase letters (A-Z)	26
Digits (0-9)	10
Special characters	24

The interpretation is as follows:

- $H < 55$: weak
- $55 \leq H < 75$: fair
- $H \geq 75$: strong

Example

For the password `ILoveProgrammingInPython3.12!` which has length $L = 31$ and contains characters from every set, we find $H = 31 \times \log_2(26+26+10+24) = 31 \times \log_2(86) \approx 199$. Since $H > 75$, this represents a **strong** password.

4 Hashing of passwords

For security reasons, passwords are never stored or transmitted as plain text on a server. To prevent a password from being intercepted, it is **hashed**. The core function of hashing is to prevent information or a file from being tampered with or misused.

4.1 What is hashing?

Hashing is a mathematical process where an input (e.g., a password) is converted into a sequence of bits or characters of a **fixed length**, the hash value. This is done with a hash function, which is always deterministic: the same input always yields the same output. An important feature of a hash function is non-reversibility: you cannot reconstruct the original password from the hash.

A good hash function has the following properties:

1. **Deterministic**: same input gives same output.
2. **Non-reversible**: impossible to go from the hash back to the password.
3. **Avalanche** effect: a small change in the input leads to a completely different hash.
4. **Low collision probability**: two different passwords should (almost) never yield the same hash.

4.2 Examples of hash functions

There are many different hash functions, from very simple to very complex. We limit ourselves here to the following two hash functions.

4.2.1 Sum of ASCII codes

Principle: sum the ASCII codes of all characters and take the result modulo a fixed value `m` (e.g., `m = 256`). Represent the result in binary with 8 bits.

Formula:

$$h = \left(\sum_{i=0}^{n} \mathrm{ord}(\mathrm{c}_i) \right) \bmod \mathrm{m}$$

with $\mathrm{ord}(\mathrm{c}_i)$ being the ASCII value of the i-th character of the password.

4.2.2 Bitshift hash

Iterative formula:

$$h_{i+1} = \big((h_i << 3) \ \oplus \ \text{ord}\,(c_i) \big) \bmod 256\,,$$

where $<< 3$ is a 3-bit left shift, and $\oplus$ is bitwise XOR.

5 Assignment Descriptions

In this section, the various assignments that make up the project are described. It is intended that a separate piece of code is written for each part of this task in `assignment1.py`, `assignment2.py`, etc.

Assignment 1

(a) Write a script that:

1. Asks the user to enter a password.
2. Checks if it has a valid form/structure (length and complexity from **Section 2.1**).
3. Repeats until the user types `"stop"`.

(b) Implement all checks from Section 2.1 in one function `isValidPW()`. Return `-1` if invalid, `0` if it is a weak password (found in the list), and `1` if it is valid and strong.

Assignment 2

Implement functions that calculate the strength score (Section 3.1) and the entropy (Section 3.2).

Assignment 3

Implement the hash functions discussed in Section 4.2 (`sumHash()` and `bitshiftHash()`).

Assignment 4

Write a function `findPasswords()` that returns a list of valid passwords that differ in only two positions but yield the same hash.

Assignment 5

This is an **open assignment** regarding advanced hash functions like Argon2, bcrypt, and scrypt. Investigate how to use these in Python and why they yield different hashes every time they are executed.